

Data Preparation Session 2 Handout

Data Import

In this course, we will suppose that data is available in flat format (no relational databases and no complex cloud architecture). Still true, data may arrive in many file formats, and knowing the most widely used formats is important. Additionally, data is not always tidy and needs to be cleaned and/or transformed. In this session, we look for data loading, and a few basic transformations.

We will use the tidyverse package, however, most of the functionalities are available in base R, typically in a more complex and less handy way. Pay attention to different capabilities and arguments of similar functions. Be aware of ChatGPT hallucinations!

Within the tidyverse library, we will use the readr and the readxl packages, for other file types, you may need the jsonlite, rvest and haven packages (see later).

csv files

Let's start with the most widely used data format, the comma separated values (.csv) file.

We can import them using read_csv:

```
library(tidyverse)
```

We import the inspections.csv to a dataset called "inspections".

Now let's try loading in the CSV file without any special settings (we suppose that the file is in your working directory):

```
inspections <- read_csv("inspections.csv")
```

And take a look at what we loaded in

```
glimpse(inspections)
```

Successful, but not the best variable names. Using variable names with space is possible but not recommended.

A possible solution: we create a vector of my names, and we use them

```
names <- c("ID", "DBAName", "AKAName", "License", "FacilityType", "Risk", "Address",  
          "City", "State", "ZIP", "InspectionDate", "InspectionType", "Results",  
          "Violations", "Latitude", "Longitude", "Location")  
inspections <- read_csv('inspections.csv', col_names=names)
```

Oups, something went wrong. We duplicated the header and destroyed data types. It would be better to read only the data.

```
inspections <- read_csv('inspections.csv', col_names=names, skip=1)  
glimpse(inspections)
```

A few more tricks will be shown later, most of the arguments are shared in the readr package (an argument with read_csv also works with read_tsv...)

If you have a semicolon (;) separated file, you can use read_csv2(). It also highlights the problem of decimal separator, a . can let us use , for separating data entries, however, we need something else if the decimal separator is a ,. An evergreen problem in IT.

tsv files

Tabulator separated values.

Let's read a tsv file from our data library.

```
# library(tidyverse) # we already loaded it, but in case
```

```
inpatient <- read_tsv('inpatient.tsv')
glimpse(inpatient)
```

We have the same problems, as before, so we solve them:

```
names <- c("DRG", "ProviderID", "Name", "Address", "City", "State",
           "ZIP", "Region", "Discharges", "AverageCharges", "AverageTotalPayments",
           "AverageMedicarePayments")
inpatient <- read_tsv('inpatient.tsv', col_names = names, skip=1)
```

Look at data types, some of them are characters, discharges seem to be integer.

We can specify the column types using the types argument of the read_() functions.

```
types = 'ccccccciicc'
inpatient <- read_tsv('inpatient.tsv', col_names = names,
                     skip=1, col_types = types)
glimpse(inpatient)
```

It works, but we have some currency amounts. Look at the \$ signs. The numbers are decimals, so we may try the double type, or the generic numeric type.

```
types = 'ccccccciidd'
inpatient <- read_tsv('inpatient.tsv', col_names = names,
                     skip=1, col_types = types)
glimpse(inpatient)
```

```
types = 'ccccccciinnn'
inpatient <- read_tsv('inpatient.tsv', col_names = names,
                     skip=1, col_types = types)
glimpse(inpatient)
```

This last one worked fine. A task that is not always evident, if you don't know your data source.

Evidently, you can also remove the \$ sign using string manipulation (covered later in this course).

read_delim()

You can extend these possibilities to any other delimited type (they are rare but exist). In `read_delim()`, you can set the `delim` argument. For the previous examples:

```
read_delim('inspections.csv', delim = ",")
```

```
read_delim('inpatient.tsv', delim = "\t")
```

Fix width files

Used typically in public administration, in hospitals, etc.

You can use the `read_fwf()` function, its arguments are not fully evident.

We need the vector of the column lengths (widths visually). How to know? From documentation, if not, by trial and error. You can leave the last length blank, but you have to specify that it is there.

```
lengths <- c(32,50,24,NA)
```

Together with the column names, a new object must be created:

```
names <- c("Name", "Title", "Department", "Salary")
```

```
widths <- fwf_widths(lengths, names)
```

And now, everything is ready for the `read_fwf()` function. Sure, we don't need the first row.

```
employees <- read_fwf('chicagoemployees.txt', widths, skip=1)  
glimpse(employees)
```

Excel files

You need the `readxl` package. It is installed with `tidyverse`, but not loaded, so don't forget to load the `readxl` library.

The `readxl` package isn't part of the core `tidyverse` so we need to load it separately

```
library(readxl)
```

If you can check the file (for example by opening it), you may take a first look to find data. Excel files tend to have multiple line headers, empty columns on the left, merged cells, and so on.

It may happen that for technical / security reasons, you are not able to open the original file, it is more challenging to find the right settings.

Arguments like *sheet*, *range*, *col_names*, *skip*, can be useful.

```
library(readxl)  
chickens <- read_excel("chickens.xlsx")  
glimpse(chickens)
```

This one worked fine; you will have more tricky ones in the exercises.

Other file formats

Almost sure that you are not the first one to import such a file. If you don't know which package you need, you can browse for it.

Let's see an example. What to do with data file with .sav extension?

Normally, it should be an SPSS data file, as SPSS stores data with this extension. Good news is that this specific data file always has the right structure (rows/columns/data types).

Web pages (web scraping):

First, you must understand the structure of a web page (html, css, nodes). It is easy if you know what you would like to import, otherwise, automatization needs advanced programming skills.

```
library(tidyverse)
library(rvest)
```

An example:

```
html <- read_html("https://en.wikipedia.org/w/index.php?title=The_Lego_Movie&oldid=998422565")
lego <- html_table(html_element(html, ".tracklist"))
lego
```

Finally, a few words about tidy data, how a good data table looks like (you can find the full paper under the Readings tab on Courses).

Tidy data rules

- Each variable forms a column.
- Each observation forms a row.
- Each type of observational unit forms a table.

Five Common Data Problems

- Column headers are values, not variable names (you see a crosstabulation / pivot table)
- Multiple variables are stored in one column
- Variables are stored in both rows and columns
- Multiple types of observational units are on the same table
- A single observational unit is in multiple tables